Text Simplification

Comparative Text Simplification using BART-Large and T5 Models

Experiments on ASSET and TurkCorpus datasets

Team #8

Members:
Jazmin Gonzalez - F11315113
Florencia Gonzalez - F11315112
Victoria Villamayor - F11315124

Datasets

TurkCorpus

→ Original sentences + multiple simplified versions
→ Focus: preserve meaning, improve clarity

ASSET

→ More diverse simplifications per sentence
→ Helps model learn varied ways to simplify

✅ **Common in simplification benchmarks**

Evaluation Metrics

SARI

→ Main metric
→ Measures Additions, Deletions, Keeps
→ Best aligned with human judgment

BLEU

→ Measures word/phrase overlap
→ Penalizes valid simplifications that use different words

FKGL

→ Readability score
→ Lower = simpler output

Domain knowledge

- Chose SARI as the primary metric, since it reflects better with how humans judge the simplification
- Removed long sentences (>100 words) since models often struggle to simplify them
- Prepocessed the data by lowercasing and trimming whitespacing, ensuring cleaner inputs

T5-Small (Text-to-Text Transformer)

Definition

- Pre-trained model.
- Lightweight version of the T5 model.
- Designed for low-resource enviroments.

Parameters

- 60 million parameters.
- 6 encoder layers.
- 6 decoder layers.

Why we chose

- Fast and easy to train.
- Offers a good balance between performance and seped.

T5-Small

Load and Split Data

```
from datasets import load_dataset
import pandas as pd
from itertools import chain

# Load ASSET validation split
dataset = load_dataset("asset", split="validation")
df = dataset.to_pandas()

# Expand complex-simple pairs
def expand_data(df):
    return pd.DataFrame(list(chain.from_iterable([
        [{"original": src, "simplification": ref} for ref in refs]
        for src, refs in zip(df["original"], df["simplifications"])
    ])))

df_train = expand_data(df[:800])
df_test = df[800:].reset_index(drop=True)
```

Load model and tokenizer

```
from transformers import T5ForConditionalGeneration, T5Tokenizer

# Load model and tokenizer
model = T5ForConditionalGeneration.from_pretrained("t5-small")
tokenizer = T5Tokenizer.from_pretrained("t5-small")
```

Training Setup

```
# Training arguments
training_args = Seq2SeqTrainingArguments(
    output_dir="./results",
    num_train_epochs=3, # times the model goes through the dataset
    per_device_train_batch_size=8, # number of training examples processes
                                   # together in 1 step per device (GPU or CPU)
    save_total_limit=1, # limits the number of saved model versions to 1
                        # (helps save disk space)
    logging_dir="./logs",
    save_steps=500, # model saves progress every 500 step
    report_to="none"
)
# Trainer
trainer = Seq2SeqTrainer(
    model=model, # training model (t5-small)
    args=training_args,
    train_dataset=tokenized_train, # preprocessed training data
    tokenizer=tokenizer
)
# Train the model
trainer.train()
```

Seq2SeqTrainer:

- Helper class from Hugging Face Transformers.
- Bulit-in evaluation during training with metrics tracking.

Text Simplification

Evaluation and Metrics

```python
# Import evaluation metrics
from easse.sari import corpus_sari
from easse.bleu import sentence_bleu
from easse.fkgl import corpus_fkgl
import numpy as np
from tqdm.auto import tqdm
import nltk
nltk.download('punkt_tab')

# filter rows with no generated simplification
df_eval = df_test.dropna(subset=["generated"]).reset_index(drop=True)

saris = []
bleus = []

for i in tqdm(range(len(df_eval))):
    sari_score = corpus_sari(
        orig_sents=[df_eval.loc[i, "original"]], # complex input
        sys_sents=[df_eval.loc[i, "generated"]], # model output
        # reference outputs
        refs_sents=[[ref] for ref in df_eval.loc[i, "simplifications"]],
        use_paper_version=True # use standard SARI formula
    )
    saris.append(sari_score)

    bleu_score = sentence_bleu(
        sys_sent=df_eval.loc[i, "generated"],
        ref_sents=df_eval.loc[i, "simplifications"]
    )
    bleus.append(bleu_score)

fkgl_score = corpus_fkgl(df_eval["generated"].tolist())
```

- SARI tells us how close the output is to good simplification behavior.
- BLEU shows how similar the sentence is to human examples.
- FKGL checks how easy the output is to read.

Output:

SARI: 34.030

BLEU: 89.992

FKGL: 7.850

Text Simplification

T5-base

- Same training arguments for both datasets
- Train for 3 epochs with a batch size of 8
- After training and testing, we computed the 3 key metrics

Model and Trainer Setup for ASSET and TurkCorpus

```
model = T5ForConditionalGeneration.from_pretrained("t5-base")

training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs",
    logging_steps=10,
    save_strategy="no",
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset
)
```

Text Simplification

T5-base

- Higher SARI score on ASSET, which suggests the model made more effective simplifications
- TurkCorpus gave a higher BLEU, which means that the outputs were similar to the original sentences

Model Output Comparison

TurkCorpus vs ASSET

Dataset	SARI	BLEU	FKGL
TurkCorpus	30.974	97.004	9.533
ASSET	37.138	92.971	8.424

Text Simplification

BART-large

- Transformer-type encoder-decoder model
- 400+ million parameters (large version)
- Fine-tuned for text simplification

Model Training with ASSET and TurkCorpus

```
training_args = TrainingArguments(
    output_dir="./results_bart",
    per_device_train_batch_size=8,
    num_train_epochs=3,
    logging_dir="./logs_bart",
    logging_steps=10,
    save_strategy="no",
    report_to="none"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset
)

trainer.train()
```

Text Simplification

Advantages of its Complexity

- Better at capturing the meaning of the original text
- Rewrites sentences more naturally and understandably
- Greater comprehension of complex grammatical structures

Results Achieved

- High SARI score → better quality in simplifications
- Outperforms simpler models in this task

Performance Comparison

Our outputs

Model	Dataset	SARI	BLEU	FKGL
T5-base	TurkCorpus	30.974	97.004	9.533
	ASSET	37.138	92.971	8.424
T5-small	TurkCorpus	31.395	95.065	9.633
	ASSET	34.030	89.992	7.850
BART-large	TurkCorpus	32.149	96.030	9.352
	ASSET	42.304	97.072	8.362

BART-large achieved a SARI score of 42.3 which is close to more complex models like MUSS and control prefixes, despite using simpler fine-tuning setup. Although our SARI and FKGL scores were slightly lower than the leaderboard, that is expected because we used a single model without control tokens.

🏆 Papers with Code leaderboard

Text Simplification on ASSET

Model	SARI	BLEU	FKGL
GPT-175B	47.94	73.92	7.73
MUSS (BART+ACCESS Supervised)	44.15	72.98	6.05
Control Prefixes (BART)	43.58	(not given)	5.97

Text Simplification on TurkCorpus

Model	SARI	BLEU	FKGL
GPT-175B	47.46	73.83	9.33
MUSS (BART+ACCESS Supervised)	42.53	78.17	7.60
Control Prefixes (BART)	42.32	(not given)	7.74

Thank you for your attention!